If you look in the gem's source, you will see that get and the three other basic HTTP methods are defined on the Resource class:

gems/rest-client-1.6.7/lib/restclient/resource.rb

```
module RestClient
  class Resource
    def get(additional_headers={}, &block) # ...
    def post(payload, additional_headers={}, &block) # ...
    def put(payload, additional_headers={}, &block) # ...
    def delete(additional_headers={}, &block) # ...
```

To make get and its siblings available in the interpreter, REST Client defines four top-level methods that delegate to the methods of a Resource at a specific URL. For example, here is how the top-level get delegates to a Resource (returned by the r method):

```
def  get(path, *args, &b)
  r[path].get(*args, &b)
end
```

You might expect to find this definition of get in the source code, together with similar definitions for put, post, and delete. However, here comes a twist. Instead of defining the four methods separately, REST Client defines all of them in one shot by creating and evaluating four *Strings of Code* *(141)* in a loop:

gems/rest-client-1.6.7/bin/restclient

```
POSSIBLE_VERBS = ['get', 'put', 'post', 'delete']

POSSIBLE_VERBS.each do |m|
  eval <<-end_eval
    def  #{m}(path, *args, &b)
        r[path].#{m}(*args, &b)
    end
  end_eval
end
```

The code above uses an exotic syntax known as a *here document*, or *heredoc* for short. What you're seeing after the eval is just a regular Ruby string, although it's not delimited by the usual quotes. Instead, it starts with a <<- sequence followed by an arbitrary *termination sequence*—in this case, end_eval. The string ends on the first line that contains only the termination sequence, so this particular string spans the lines from the def to the first end included. The code uses regular string substitution to generate and eval four Strings of Code, one each for the definitions of get, put, post, and delete.

Most Strings of Code feature some kind of string substitution, as in the example above. For an alternate way to use eval, you can evaluate arbitrary